SentinelIQ 03 — Non-Functional Requirements & Security

Oct 13, 2026 · @maxi

SentinelIQ treats its own security, auditability and reproducibility as requirements, because a security tool that can be attacked or cannot explain itself has no value.

Security requirements

ID	Control	Requirement
NFR-S1	Authentication	JWT with short-lived access tokens; passwords stored with a slow hash (bcrypt or argon2)
NFR-S2	Authorization	Role-based access on every route; deny by default
NFR-S3	Secrets	No secrets in the repository; environment variables locally, a secret manager in the cloud
NFR-S4	Input handling	Every API input validated against a schema; parameterized queries only
NFR-S5	Rate limiting	Login and ingestion endpoints limited per user and per IP
NFR-S6	Dependencies	Automated dependency and container image scanning in CI
NFR-S7	Transport	HTTPS only in deployed environments

Prompt-injection controls

Log text is written by outsiders, and an attacker can hide instructions inside it. Because SentinelIQ sends log content to an LLM, it treats all event text as untrusted data.

- **Separate instructions from data:** the system prompt holds the rules; event content goes in a clearly delimited data section and is never concatenated into instructions.
- **Constrain the output:** the LLM must return a structured result (technique IDs, summary, confidence) that is validated against a schema before storage.

- **Ground every claim:** an explanation may only cite techniques that retrieval returned; anything else is discarded.
- **No tool access:** the LLM cannot call APIs, run commands or change data, so a successful injection can at worst produce a wrong explanation.
- **Test it:** include a set of malicious log lines in the evaluation suite and report how many the system resists.

Reliability

- **Idempotent ingestion:** every event carries a deduplication key, so retries and duplicate sends never create duplicate rows.
- **Graceful degradation:** if the LLM or retrieval service is unavailable, events are still scored and queued, and the explanation is marked pending.
- **No silent failures:** a failed scoring or triage job is recorded with its error and retried a limited number of times.
- **Health checks:** the API, database and worker each expose a health endpoint that the container platform uses.
- **Data integrity:** the audit log is append-only; database migrations are versioned and run automatically.

Performance targets

These numbers are proposals. Phase 7 measures them on a laptop-class machine running the Docker setup and records the real results, whether or not they meet the target.

ID	Metric	Proposed target
NFR-P1	Ingestion API response time	95% of requests under 300 ms
NFR-P2	Time from ingestion to a stored risk score	Under 5 seconds at demo load
NFR-P3	Explanation generation per flagged event	Under 10 seconds
NFR-P4	Alert queue page load with 100,000 stored events	Under 2 seconds

ID	Metric	Proposed target
NFR-P5	Full stack startup from a clean checkout	Under 10 minutes including image build

Operability and reproducibility

- **One command:** `docker-compose up` starts the API, database, worker and frontend, with seed data available through a second documented command.
- **Versioned models:** each trained model is saved with its version, training data reference, parameters and metrics, and every score records the model version that produced it.
- **Structured logging:** JSON logs with a request ID that follows an event from ingestion to decision.
- **Configuration:** all settings come from environment variables, with a documented example file and no defaults that are unsafe in production.
- **Automation:** CI runs lint, tests and a container build on every pull request; deployment happens only from the main branch.

Explainability and audit

Any analyst, lead or reviewer must be able to reconstruct why an alert looked the way it did. The system stores the following for each stage.

Stage	Stored with every record
Score	Risk value, model version, input features used, timestamp
Explanation	Text, retrieved ATT&CK technique IDs and passages, prompt version, LLM model name, timestamp
Decision	User, action, required note, timestamp

Audit rows are append-only: the API offers no update or delete route for them, and database permissions for the application account enforce the same rule.